DjOpenKB Deployment Guide

This guide provides the command sequence for preparing, deploying, starting, updating, and troubleshooting DjOpenKB.

Replace placeholder values such as `<SERVER-IP-OR-DOMAIN>`, `<AD-SERVER-IP>`, `<AD-DOMAIN>`, `<NETBIOS-DOMAIN>`, `<SERVICE-ACCOUNT>`, `<POSTGRES_PASSWORD>`, and `<AI_API_KEY>` with your own deployment values.

---

1. Important Values to Change Before Deployment

Before running the first deployment, review these files carefully.

`.env` — non-secret deployment configuration

Use `.env` for server, Django, LDAP/AD, and OpenKB path settings. Do **not** store passwords or API keys here.

```
# Django / website access
DJANGO_DEBUG=false
DJANGO_ALLOWED_HOSTS=<SERVER-IP-OR-DOMAIN>,localhost,127.0.0.1
CSRF_TRUSTED_ORIGINS=https://<SERVER-IP-OR-
DOMAIN>:8080,https://localhost:8080,https://127.0.0.1:8080

# Database connection inside Docker
POSTGRES_DB=djopenkb
POSTGRES_USER=djopenkb
POSTGRES_HOST=db
POSTGRES_PORT=5432
USE_SQLITE=false

# Vault inside Docker
VAULT_ADDR=http://vault:8200
VAULT_SECRET_PATH=secret/djopenkb

# AD / LDAP settings
LDAP_ENABLED=true
LDAP_SERVER_URI=ldap://<AD-SERVER-IP>:389
LDAP_BIND_DN=<SERVICE-ACCOUNT>@<AD-DOMAIN>
LDAP_AD_DOMAIN=<AD-DOMAIN>
LDAP_NETBIOS_DOMAIN=<NETBIOS-DOMAIN>
LDAP_USER_SEARCH_BASE=DC=<DOMAIN-PART-1>,DC=<DOMAIN-PART-2>
LDAP_USER_FILTER=(|(userPrincipalName=%(user)s)(sAMAccountName=%(user)s)(mail=%
(user)s))
LDAP_ALLOWED_EMAIL_DOMAINS=<AD-DOMAIN>

# Integrated OpenKB paths
OPENKB_BASE_DIR=OpenKB-main
OPENKB_DATA_DIR=openkb-data
```

Example for a lab domain `openkb.local`:

```
LDAP_SERVER_URI=ldap://<AD-SERVER-IP>:389
LDAP_BIND_DN=svc_djopenkb@openkb.local
LDAP_AD_DOMAIN=openkb.local
LDAP_NETBIOS_DOMAIN=OPENKB
LDAP_USER_SEARCH_BASE=DC=openkb,DC=local
LDAP_ALLOWED_EMAIL_DOMAINS=openkb.local
```

`vault/bootstrap/djopenkb.env` — temporary secret seed file

This file is used only for first-time Vault seeding or intentional secret rotation. Do **not** commit it to GitHub.

```
DJANGO_SECRET_KEY="<LONG_RANDOM_DJANGO_SECRET_KEY>"
POSTGRES_PASSWORD="<STABLE_POSTGRES_PASSWORD>"

# LDAP service account password
LDAP_BIND_PASSWORD="<SERVICE_ACCOUNT_PASSWORD>"

# AI provider key used by OpenKB / LiteLLM
LLM_API_KEY="<AI_API_KEY>"

# Optional provider-specific aliases if used by your deployment
GEMINI_API_KEY="<GEMINI_API_KEY_IF_USING_GEMINI>"
OPENAI_API_KEY="<OPENAI_API_KEY_IF_USING_OPENAI>"
ANTHROPIC_API_KEY="<ANTHROPIC_API_KEY_IF_USING_ANTHROPIC>"
```

Important:

- Keep `POSTGRES_PASSWORD` stable after PostgreSQL has been initialized.
- If you change `POSTGRES_PASSWORD` in Vault after the database already exists, Django may fail to connect until the password is also changed inside PostgreSQL.
- Use quotes around secret values, especially when the value contains special characters.
- After Vault confirms the secret is seeded, remove `vault/bootstrap/djopenkb.env` from the server folder.

---

2. Prerequisites Before First `docker compose up`

Before starting the stack for the first time, make sure these are ready:

```
1. Docker and Docker Compose plugin are installed.
2. Runtime folders exist for Vault, PostgreSQL, OpenKB, and Nginx certificates.
3. Nginx HTTPS certificate/key are generated before Nginx starts.
4. `.env` contains deployment settings such as hostnames, AD server IP, AD domain,
and OpenKB paths.
5. `vault/bootstrap/djopenkb.env` contains real secrets such as Django secret key,
```

```
Postgres password, LDAP bind password, and AI API keys.
6. OpenKB source is already integrated locally under `OpenKB-main/`.
7. OpenKB workspace folder exists under `openkb-data/`.
```

For a Docker deployment, you do **not** need a host `.venv` just to run OpenKB. OpenKB is already downloaded and integrated inside the project as `OpenKB-main/`. The recommended method is to run OpenKB commands inside the `web` container.

Important OpenKB rule:

```
OpenKB init must be completed before the integrated AI/chatbox can use OpenKB.
```

If OpenKB has not been initialized, the chatbox may warn that OpenKB must be initialized first. The fix is to run `openkb init` inside the `openkb-data` directory, then sync DjOpenKB articles into OpenKB.

Standard first-time sequence:

```
Generate Nginx HTTPS certificate/key
→ prepare .env
→ prepare vault/bootstrap/djopenkb.env
→ docker compose up -d --build
→ run migrations / create superuser
→ run openkb init inside /app/openkb-data
→ run sync_openkb_ai
→ remove vault/bootstrap/djopenkb.env
```

---

3. OpenKB AI Initialization and Model Format

DjOpenKB integrates the downloaded OpenKB source from:

```
https://github.com/VectifyAI/OpenKB
```

The integrated source is stored at:

```
OpenKB-main/
```

The local OpenKB workspace/data folder is:

```
openkb-data/
```

When OpenKB is initialized, it creates:

```
openkb-data/.openkb/config.yaml
```

Required OpenKB init command for Docker deployment

Run this **after** `docker compose up -d --build`, because the `web` container must exist first:

```
# Initialize OpenKB from inside the web container
# This command must be run from /app/openkb-data so OpenKB creates openkb-
data/.openkb/config.yaml
docker compose exec web sh -lc "cd /app/openkb-data && PYTHONPATH=/app/OpenKB-main
python -m openkb.cli init"
```

During `openkb init`, select the AI provider/model that matches the API key stored in Vault.

OpenKB uses LiteLLM-style model names:

```
OpenAI:
  gpt-5.4
  gpt-5.4-mini
  OpenAI models can usually omit the provider prefix.

Gemini:
  gemini/gemini-2.5-flash
  gemini/gemini-3.1-pro-preview
  Format: gemini/<model-name>

Anthropic:
  anthropic/claude-sonnet-4-6
  anthropic/claude-opus-4-6
  Format: anthropic/<model-name>

Other LiteLLM-supported providers:
  <provider>/<model-name>
```

Example `openkb-data/.openkb/config.yaml`:

```
model: gemini/gemini-2.5-flash
language: en
pageindex_threshold: 20
```

API key alignment

The provider chosen during `openkb init` must match the API key stored in Vault.

```
Gemini model selected:
  vault/bootstrap/djopenkb.env should contain GEMINI_API_KEY and/or LLM_API_KEY

OpenAI model selected:
  vault/bootstrap/djopenkb.env should contain OPENAI_API_KEY and/or LLM_API_KEY

Anthropic model selected:
  vault/bootstrap/djopenkb.env should contain ANTHROPIC_API_KEY and/or LLM_API_KEY
```

Recommended approach:

- Keep API keys in Vault through `vault/bootstrap/djopenkb.env` or `vault kv patch`.
- Do not commit `openkb-data/.env` if OpenKB creates one.
- If OpenKB prompts for an API key during init, only enter it if you understand it may be written into the OpenKB workspace. Otherwise, prefer storing the key in Vault and exposing it through the container environment.

After OpenKB init, run:

```
# Sync DjOpenKB articles into OpenKB AI
docker compose exec web python manage.py sync_openkb_ai
```

If the chatbox still warns that OpenKB is not initialized, check:

```
# Confirm OpenKB config exists
docker compose exec web sh -lc "ls -la /app/openkb-data/.openkb && cat
/app/openkb-data/.openkb/config.yaml"
```

---

4. First-Time Linux Server Setup

Run these commands on the Linux server that will host DjOpenKB.

```
# Update package index before installing dependencies
sudo apt update

# Install Git, Docker, Docker Compose plugin, and OpenSSL for Nginx HTTPS cert
generation
sudo apt install -y git docker.io docker-compose-plugin openssl

# Enable Docker to start automatically after server reboot
sudo systemctl enable docker

# Start Docker now
sudo systemctl start docker
```

```
# Optional: allow the current user to run Docker without sudo
sudo usermod -aG docker $USER

# Apply the Docker group change without logging out
newgrp docker

# Clone the DjOpenKB repository from GitHub
git clone https://github.com/ErinFlyingSkyRocket/DjOpenKB.git

# Enter the project directory
cd DjOpenKB

# Create required runtime folders for Vault, PostgreSQL, Nginx certs, and OpenKB
data
mkdir -p vault/bootstrap vault/file vault/keys vault/logs postgres-data openkb-
data/raw openkb-data/wiki nginx/certs

# Generate the local HTTPS certificate/key used by Nginx
bash nginx/certs/generate-localhost-cert.sh

# Copy the Vault bootstrap example file into the real bootstrap secret file
cp vault/bootstrap/djopenkb.env.example vault/bootstrap/djopenkb.env

# Edit Vault bootstrap secrets: Django secret key, Postgres password, LDAP
password, and AI API keys
nano vault/bootstrap/djopenkb.env

# Edit deployment settings: allowed hosts, CSRF origins, AD server IP/domain, and
OpenKB paths
nano .env

# Build and start all Docker services for the first time
# This starts Vault, seeds secrets, starts PostgreSQL, Django web, Nginx, and the
cleanup scheduler
docker compose up -d --build

# Check that containers are running or healthy
docker compose ps

# Run Django database migrations
docker compose exec web python manage.py migrate

# Create the first local Django administrator account
docker compose exec web python manage.py createsuperuser

# Initialize OpenKB inside the downloaded local OpenKB integration
# This creates openkb-data/.openkb/config.yaml
# Choose the AI provider/model when prompted
docker compose exec web sh -lc "cd /app/openkb-data && PYTHONPATH=/app/OpenKB-main
python -m openkb.cli init"

# Sync DjOpenKB articles into the OpenKB AI data store
docker compose exec web python manage.py sync_openkb_ai
```

```
# Remove the plaintext Vault bootstrap file after Vault confirms the secret is
seeded
rm vault/bootstrap/djopenkb.env
```

Open the website:

```
https://<SERVER-IP-OR-DOMAIN>:8080
```

For local testing:

```
https://localhost:8080
https://127.0.0.1:8080
```

A self-signed certificate warning is expected unless you replace the certificate with one trusted by your environment.

---

5. First-Time Windows Docker Desktop Setup

Use this only when deploying or testing from Windows with Docker Desktop.

```
# Clone the DjOpenKB repository from GitHub
git clone https://github.com/ErinFlyingSkyRocket/DjOpenKB.git

# Enter the project directory
cd DjOpenKB

# Create required runtime folders
New-Item -ItemType Directory -Force vault\bootstrap, vault\file, vault\keys,
vault\logs, postgres-data, openkb-data\raw, openkb-data\wiki, nginx\certs

# Generate the local HTTPS certificate/key for Nginx
powershell -ExecutionPolicy Bypass -File .\nginx\certs\generate-localhost-cert.ps1

# Copy the Vault bootstrap example into the real bootstrap file
Copy-Item vault\bootstrap\djopenkb.env.example vault\bootstrap\djopenkb.env

# Edit Vault secrets
notepad vault\bootstrap\djopenkb.env

# Edit main environment settings
notepad .env

# Start and build all Docker services
docker compose up -d --build
```

```
# Check container status
docker compose ps

# Run Django migrations
docker compose exec web python manage.py migrate

# Create local Django administrator account
docker compose exec web python manage.py createsuperuser

# Initialize OpenKB inside /app/openkb-data before using the AI/chatbox
# Choose the AI provider/model when prompted
docker compose exec web sh -lc "cd /app/openkb-data && PYTHONPATH=/app/OpenKB-main
python -m openkb.cli init"

# Sync DjOpenKB articles into OpenKB AI
docker compose exec web python manage.py sync_openkb_ai

# Remove plaintext Vault bootstrap file after successful seeding
Remove-Item .\vault\bootstrap\djopenkb.env
```

---

6. OpenKB Reinitialization / Maintenance

Use this when the chatbox warns that OpenKB is not initialized, or when `openkb-data/.openkb/config.yaml` is missing.

```
# Initialize OpenKB workspace/config inside openkb-data
docker compose exec web sh -lc "cd /app/openkb-data && PYTHONPATH=/app/OpenKB-main
python -m openkb.cli init"

# Confirm OpenKB config was created
docker compose exec web sh -lc "ls -la /app/openkb-data/.openkb && cat
/app/openkb-data/.openkb/config.yaml"

# Sync DjOpenKB articles into OpenKB AI
docker compose exec web python manage.py sync_openkb_ai
```

If you need to change OpenKB model/provider later:

```
# Edit OpenKB model configuration
nano openkb-data/.openkb/config.yaml

# Update the matching API key in Vault if needed
docker compose exec vault vault login
docker compose exec vault vault kv patch secret/djopenkb LLM_API_KEY="
<NEW_API_KEY>"

# Restart services that read Vault secrets
docker compose restart web cleanup-scheduler
```

```
# Re-sync articles into OpenKB AI
docker compose exec web python manage.py sync_openkb_ai
```

Optional host `.venv` method

Use this only if you intentionally want to run OpenKB directly on the Linux host instead of inside Docker.

```
# Create and activate a local Python virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Install project dependencies locally
pip install --upgrade pip
pip install -r requirements.txt

# Run OpenKB init from the host using the downloaded OpenKB source
mkdir -p openkb-data
cd openkb-data
PYTHONPATH=../OpenKB-main python -m openkb.cli init
cd ..

# Deactivate the host virtual environment when finished
deactivate
```

---

7. Normal Subsequent Startup

Use this after first-time setup is already completed.

```
# Enter the project folder
cd DjOpenKB

# Start existing containers using existing Vault/Postgres/OpenKB data
docker compose up -d

# Check running container status
docker compose ps
```

Do not recreate `vault/bootstrap/djopenkb.env` for normal startup. Vault should already contain the stored secrets.

---

8. Normal Shutdown

```
# Stop containers while keeping persistent Vault/Postgres/OpenKB data
```

```
docker compose down
```

Avoid this unless intentionally resetting everything:

```
# Dangerous: removes Docker-managed volumes and may reset persistent data
docker compose down -v
```

---

9. Updating After Git Pull

```
# Enter the project folder
cd DjOpenKB

# Pull latest code from GitHub
git pull

# Rebuild and recreate services that depend on updated code
docker compose up -d --build --force-recreate web nginx cleanup-scheduler

# Run migrations after code updates
docker compose exec web python manage.py migrate

# Re-sync OpenKB AI if article handling or AI sync logic changed
docker compose exec web python manage.py sync_openkb_ai
```

---

10. Restart Common Services

```
# Restart only Django web container
docker compose restart web

# Restart web and nginx after template/static/login-page changes
docker compose restart web nginx

# Rebuild web and nginx if Python/templates/settings changed
docker compose up -d --build --force-recreate web nginx

# Restart cleanup scheduler only
docker compose restart cleanup-scheduler
```

---

11. Vault Secret Management

```
# Check Vault logs
docker compose logs vault --tail=100

# Re-run Vault init after intentionally updating vault/bootstrap/djopenkb.env
docker compose up --force-recreate vault-init

# Restart services that read secrets from Vault
docker compose restart web cleanup-scheduler

# Login to Vault CLI using root/admin token
docker compose exec vault vault login

# Patch LDAP bind password directly in Vault
docker compose exec vault vault kv patch secret/djopenkb LDAP_BIND_PASSWORD="
<NEW_LDAP_BIND_PASSWORD>"

# Patch AI provider key directly in Vault
docker compose exec vault vault kv patch secret/djopenkb LLM_API_KEY="
<NEW_API_KEY>"
```

Do not change `POSTGRES_PASSWORD` in Vault after PostgreSQL has already been initialized unless you also update the password inside PostgreSQL.

---

12. PostgreSQL Commands

```
# View PostgreSQL logs
docker compose logs db --tail=100

# Open PostgreSQL shell
docker compose exec db psql -U djopenkb -d djopenkb

# Backup database to local file
docker compose exec db pg_dump -U djopenkb djopenkb > backup_djopenkb.sql

# Restore database from local file
cat backup_djopenkb.sql | docker compose exec -T db psql -U djopenkb -d djopenkb
```

---

13. Django Commands

```
# Run migrations
docker compose exec web python manage.py migrate

# Create local Django superuser
docker compose exec web python manage.py createsuperuser
```

```
# Open Django shell
docker compose exec web python manage.py shell

# Collect static files manually
docker compose exec web python manage.py collectstatic --noinput
```

---

14. AD/LDAP Checks

```
# Test if web container can reach AD LDAP port
docker compose exec web python -c "import socket; s=socket.socket();
s.settimeout(5); s.connect(('<AD-SERVER-IP>',389)); print('LDAP 389 reachable');
s.close()"

# Print active LDAP settings from Django
docker compose exec web python manage.py shell -c "from django.conf import
settings; print(settings.AUTH_LDAP_SERVER_URI); print(settings.AUTH_LDAP_BIND_DN);
print(bool(settings.AUTH_LDAP_BIND_PASSWORD));
print(settings.AUTH_LDAP_USER_SEARCH)"

# Test LDAP bind and user search
docker compose exec web python manage.py test_ldap_auth <username>

# Test LDAP authentication with password prompt
docker compose exec web python manage.py test_ldap_auth <username> --auth
```

---

15. MFA Checks

```
# Check whether MFA model/table exists through Django shell
docker compose exec web python manage.py shell -c "from kb.models import
UserMFADevice; print(UserMFADevice.objects.count())"

# Run migrations if MFA table was newly added
docker compose exec web python manage.py migrate

# Open Django shell for MFA reset
docker compose exec web python manage.py shell
```

Inside Django shell:

```
from django.contrib.auth import get_user_model
from kb.models import UserMFADevice

User = get_user_model()
```

```
user = User.objects.get(username="<username>")
UserMFADevice.objects.filter(user=user).delete()
```

---

16. Logs and Troubleshooting

```
# View all service logs
docker compose logs -f

# View web logs only
docker compose logs -f web

# View nginx logs only
docker compose logs -f nginx

# View Vault init logs
docker compose logs vault-init --tail=100

# View container status
docker compose ps
```

---

17. Dangerous Reset Commands

Only use these when intentionally resetting local development data.

```
# Stop containers and remove Docker-managed volumes
docker compose down -v

# Delete local PostgreSQL bind-mounted data
rm -rf postgres-data

# Delete local Vault file storage and keys
rm -rf vault/file vault/keys

# Delete local OpenKB generated data
rm -rf openkb-data/raw openkb-data/wiki openkb-data/.openkb
```

After using reset commands, recreate runtime folders and seed Vault again using `vault/bootstrap/djopenkb.env`.